

MODUL 5

Dein Agent lernt deinen Workflow.

Spec rein, Struktur übernehmen, erstes Feature raus.

DOZENT

Prof. Dr.-Ing. Jacek Zawisza

PROFESSUR

Betriebliche Informationssysteme & Digitalisierung · FB 3

WAS WIR HEUTE MACHEN.

01

Recap & Stand

Setup steht — wo fangen wir an?

02

spec.md ins Repo

Die Grundlage, aus der alles entsteht.

03

Der modus-operandi-Move

Ein Prompt — und das Projekt hat Struktur.

04

Spec → Backlog

Anforderungen werden abhakbare Features.

05

Architektur

Framework frei, Backend Pflicht.

06

Versionskontrolle & Doku

Nach jedem Feature: dokumentieren, committen, pushen.

07

Erstes Feature — live

Einmal die ganze Schleife gemeinsam.

08

Abgabe & Bewertung

Wie dein Repo zählt — und die Spielregel.

01

ABSCHNITT

Recap & Stand.

Setup steht — jetzt bauen wir das Projekt auf.

KURZER CHECK, BEVOR WIR BAUEN

Drei Dinge sollten aus letzter Woche stehen.

01

KI-AGENT LÄUFT

Codex (ChatGPT), der Kurs-Pool (Aider) oder dein eigenes Abo — Hauptsache, er antwortet im Terminal.

02

REPO STEHT

Ein privates GitHub-Repo, angelegt mit
`gh repo create`.

03

SPEC.MD LIEGT BEREIT

Deine fertige Spezifikation aus der Discovery — die vier Bausteine, sauber.

Hängt jemand beim Setup? Jetzt kurz Hand heben — wir fixen das in zwei Minuten. Nicht die ganze Stunde dran verlieren.

SO KOMMST DU IN DEN POOL

Persönlicher Account — im Browser einloggen, dann deinen Key erzeugen.

- 1 Browser öffnen** models.prof-z.com/ui aufrufen.
- 2 Einloggen** Deine offizielle HTW-Studi-E-Mail (im Format vorname.nachname@student.htw-berlin.de – prüfe im Zweifel deine E-Mail in Moodle) + Passwort "smart-apps-htw".
- 3 Key erzeugen** „Virtual Keys“ → „+ Create New Key“ → Model "smart-app" → erstellen.
- 4 Kopieren** Den sk-...-Key sofort kopieren — er wird nur einmal gezeigt.

Anderer Login als die Kunden-Chats: models.prof-z.com ist nur das Coding-Modell — nicht customer.prof-z.com.

KEIN CHATGPT-ABO? DER KURS-POOL

Dasselbe Codex wie sonst — nur ein anderes Profil. Persönlicher Account, eigenes Budget.

PROFIL · ~/.codex/smart-apps.config.toml

```
model = "smart-apps"
model_provider = "smart-apps"
[model_providers.smart-apps]
base_url = "https://models.prof-
z.com/v1"
env_key = "SMART_APPS_API_KEY"
wire_api = "responses"
```

KEY SETZEN & STARTEN

```
export SMART_APPS_API_KEY=sk-...
# dein Key aus dem Login (Folie
davor)
codex --profile smart-apps
# gewohnte TUI, DeepSeek im Backend
```

Codex hast du in Modul 4 installiert · Budget pro Person, Workflow identisch zum ChatGPT-Pfad.

02

ABSCHNITT

spec.md ist die Grundlage.

Alles, was der Agent baut, kommt hier raus.

ERST DIE SPEC, DANN DER AGENT

Die spec.md ist dein Vertrag mit dem Agenten — er liest sie zuerst.

SPEC INS REPO LEGEN & SICHERN

```
mkdir -p docs  
  
# deine spec.md aus der Discovery nach docs/ kopieren  
  
git add docs/spec.md  
  
git commit -m "docs: spec.md aus der Discovery"
```

Garbage in, garbage out: Eine vage Spec macht eine vage App. Was hier unklar ist, rät der Agent — selten so, wie du es meinst.

03

ABSCHNITT

Dein Agent lernt deinen Workflow.

Ein Prompt — und das Projekt hat Struktur.

WAS IST MODUS-OPERANDI?

modus-operandi

deine Arbeitsweise, als Repo dokumentiert

Eine Methodik für KI-natives Bauen: eine feste Doku-Struktur (CLAUDE.md, docs/), aus der der Agent jederzeit Kontext, Stand und Regeln zieht. Leitsatz: Wenn es nicht aufgeschrieben ist, existiert es nicht.

DER CLOU

Du legst die Struktur nicht selbst an.

Der Agent liest mein Repo und übernimmt sie für dein Projekt — mit einem Prompt.

DER EINE PROMPT, DER ALLES AUFSETZT

Tippst du genau so in deinen Agenten — das Repo strukturiert sich dann selbst.

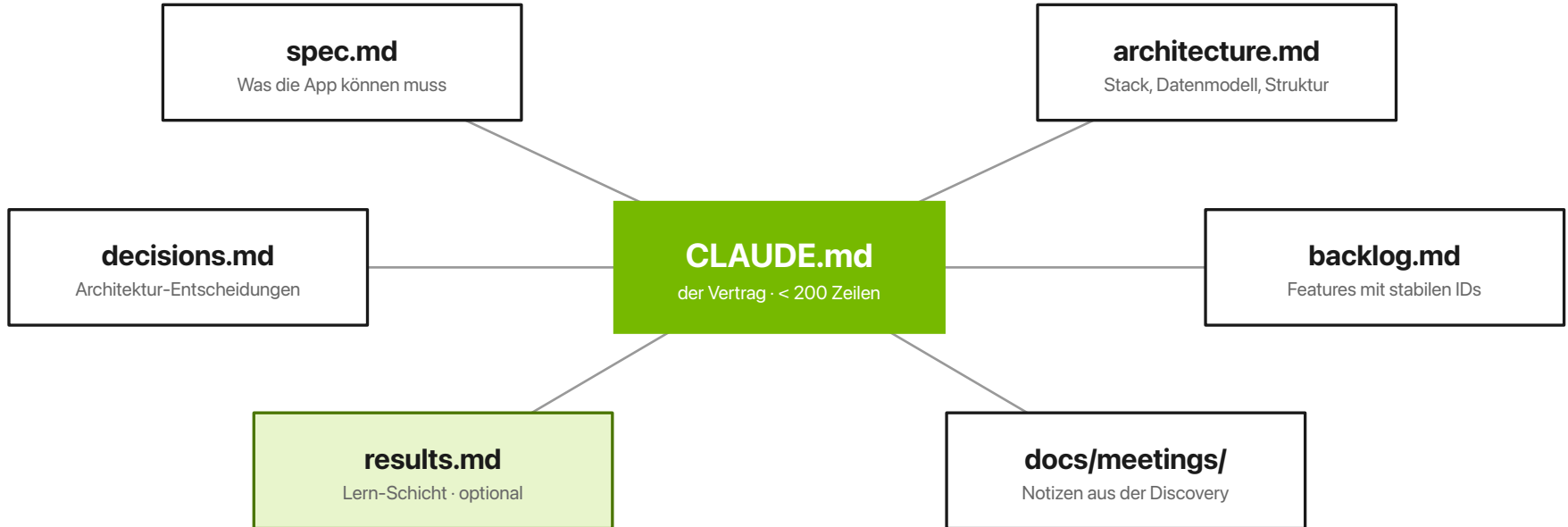
PROMPT AN DEINEN AGENTEN

Lies das Repo github.com/jacekzawisza/modus-operandi und richte dieses Projekt nach dieser Methodik als SOLO-Projekt ein — Doku-Struktur, CLAUDE.md bzw. AGENTS.md, docs/. Nimm meine docs/spec.md als Grundlage. Bitte ignoriere die prd.md. Ihre Rolle wird von der spec.md eingenommen. Da wir ein Soloprojekt sind, brauchen wir auch keine Meetings-und Results-ordner.

Was dabei entsteht — die Artefakt-Landschaft — siehst du gleich. spec.md hast du schon abgelegt; der Agent ergänzt den Rest drumherum.

DIE ARTEFAKT-LANDSCHAFT

CLAUDE.md ist der Vertrag — alle anderen Dateien hängen daran. Der Agent liest sie zuerst.



i.A.a. *Modus Operandi* (Zawisza 2026) · github.com/jacekzawisza/modus-operandi

04

ABSCHNITT

Vom Spec zum Backlog.

Anforderungen werden zu abhakbaren Features.

FEATURES MIT NUMMERN

Du kürzt: nicht alles muss in Phase 1 — der Agent überbaut gern. Erst das Skelett, das läuft.

PROMPT AN DEINEN AGENTEN

Brich meine docs/spec.md in ein docs/backlog.md auf: jede Anforderung als Feature mit ID, Phase und Status. Sortiere nach Phasen — Kern zuerst.

1

PHASE 1 — KERN

Gäste, Tische, Reservierungen anlegen und speichern. Das Skelett.

2

PHASE 2 — REGELN

2-Stunden-Sperre, Gruppenmenü ab 8, Stammgast-Rabatt.

3

PHASE 3 — KOMFORT

Übersichten, Filter, zweiter Standort, Catering.

05

ABSCHNITT

Architektur & Datenbank.

Framework frei — die Datenbank steht.

FRAMEWORK FREI, SQLITE ALS DATENBANK

DEINE WAHL — FRAMEWORK

Next.js ist der naheliegende Weg.

React, läuft sauber, Codex baut es flüssig. Bleib bei JS — dann hilft der Agent am besten.

STANDARD — DATENBANK

SQLite — eine Datei, kein Server.

Null Setup, lokal sofort lauffähig, der Agent richtet alles ein. Warum nicht Postgres/Supabase? Nächste Folie.

PROMPT: „Setz für das Projekt in dem Ordner ein Next.js-Projekt mit SQLite (Prisma) auf und halte die Wahl in docs/architecture.md fest.“

WARUM SQLITE — NICHT POSTGRES ODER SUPABASE?

Wir bauen lokal und prüfen den Code. Da gewinnt das Einfachste.

01

SQLITE — DER STANDARD

Eine Datei, kein Server. Der Agent richtet's ein, danach kann nichts haken. Lokal sofort lauffähig.

02

POSTGRESQL (LOKAL)

Echter DB-Server, läuft als Dienst. Mehr Fehlerquellen — Port, Service, Plattform — auch mit Agent.

03

SUPABASE

Gehostet, aber viel manuelle Web-Einrichtung. Brauchten wir für die alte Online-Abnahme — die ist weg.

Kein Lock-in: Mit Prisma ist der Wechsel zu Postgres später eine Zeile. Du verlierst durch SQLite nichts.

06

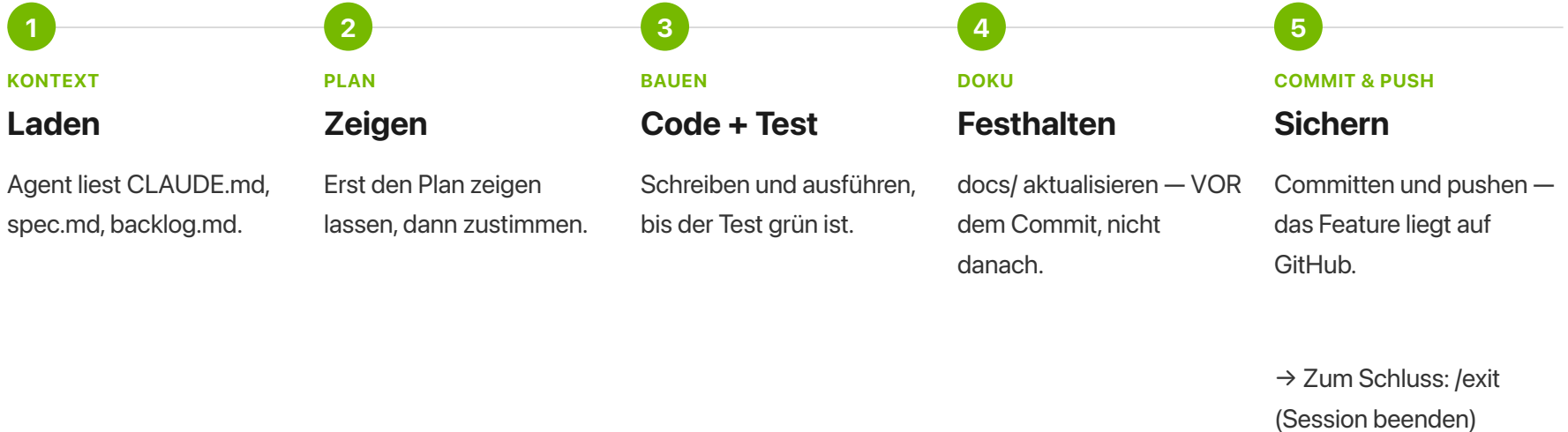
ABSCHNITT

Versionskontrolle & Doku.

Nach jedem Feature: dokumentieren, committen, pushen.

DIE SESSION-SCHLEIFE

Fünf Schritte — Schritt 4 vergisst fast jeder.



VOR JEDEM COMMIT: DOKUMENTIEREN LASSEN

Nicht dokumentiert = existiert nicht. Das liest nächste Session der Agent — und du.

PROMPT AN DEINEN AGENTEN

Bevor wir committen: dokumentiere, was sich geändert hat — Entscheidungen in docs/decisions.md, den Feature-Status in docs/backlog.md.

DECISIONS.MD

Warum so gebaut? Jede Architektur-Entscheidung, ein Satz.

BACKLOG.MD

Feature von todo → done, mit Commit-Bezug.

CLAUDE.MD

Neue Konvention oder Stolperstein? Hier rein.

NACH JEDEM FEATURE: SICHERN

Ein Feature fertig = ein Commit + ein Push. Nicht erst am Ende.

DER RHYTHMUS — NACH JEDEM FEATURE

```
# 1) Doku aktualisieren lassen (decisions.md, backlog.md)
git add .
git commit -m "feat: KP-001 Gäste anlegen"
git push
```

Push heißt: dein Code liegt auf GitHub, nicht nur lokal — Laptop kaputt, alles noch da. Und die Doku gehört in denselben Commit, nicht „später“.

07

ABSCHNITT

Erstes Feature — live.

Einmal die ganze Schleife gemeinsam.

WIR BAUEN FEATURE KP-001

Aus dem Backlog: „Gäste anlegen und speichern.“

1

Auftrag

„Bau KP-001 aus docs/backlog.md.“ Klein und konkret — nicht „bau die App“.

2

Plan ansehen

Was will der Agent anfassen? Erst lesen, dann zustimmen.

3

Bauen & prüfen

Agent schreibt und testet. Du prüfst das Ergebnis — nicht jede Zeile.

4

Dokumentieren

decisions.md + backlog.md aktualisieren lassen. Der oft vergessene Schritt.

5

Committen & pushen

git commit + git push — das Feature liegt auf GitHub. Dann das nächste.

08

ABSCHNITT

Abgabe & Bewertung.

Wie dein Repo zählt — und die Spielregel.

Deine Hausaufgabe bis nächste Woche

WAS DU JETZT KANNST

Repo nach modus-operandi, ein `backlog.md`, und du hast eine ganze Feature-Schleife einmal durch.

Ab hier wiederholst du nur noch.

1 · PHASE 1 BAUEN

Feature für Feature aus dem Backlog — immer die ganze Schleife, immer klein.

2 · JEDE SESSION DOKUMENTIEREN

Vor jedem Commit: `decisions.md` + `backlog.md` aktualisieren lassen.

3 · SAUBER COMMITTEN

Pro Feature ein Commit mit ID. Deine Historie erzählt die Geschichte.

WIE WIRD BEWERTET?

50 Punkte, individuell — dein GitHub-Repo ist der Beweis. Kein Deployment, kein Pitch.

15

SPEC-QUALITÄT

Wie sauber ist deine spec.md gegen das Kundenmodell?

20

MODELL-ABNAHME

Datenmodell + Geschäftsregeln im Code, per KI-Review gegen das versteckte Modell geprüft.

10

KALIBRIERUNG

Schätzt du selbst richtig ein, was funktioniert — und was nicht?

5

PROZESS & DOKU

Saubere commit-Historie, gepflegte docs/ — modus-operandi gelebt.

Dein Repo bleibt: technische Expertise statt Präsentation — und ein Portfolio-Stück für spätere Bewerbungen.

WAFFENGLEICHHEIT: KI PRÜFT KI

Wenn ihr mit KI baut, prüfe ich mit KI. Fair in beide Richtungen.

MIT EINWILLIGUNG — DER STANDARD

Du baust mit KI, ich prüfe mit KI.

In Moodle das Kästchen ankreuzen (Datenschutz). Abgabe des GitHub-Repos als Link über Moodle, wie besprochen.

OHNE EINWILLIGUNG

Dann händisch — auf beiden Seiten.

Kein KI-Einsatz beim Bauen, Code per E-Mail, manuelle Prüfung.
Konsequent gleich.

Ohne Einwilligung keine Einreichung über Moodle. Die KI-Prüfung ist Hilfsmittel — die Note bestätige ich.

SO REICHST DU EIN

Nur der Link zu deinem GitHub-Repo — über Moodle. Kein Upload, keine Dateien.

NUR DER REPO-LINK

Die GitHub-URL ins Moodle-Textfeld. Keine Dateien, kein Zip.

ZUGRIFF

Öffentlich — oder privat + @jacekzawisza als Collaborator. Was ich nicht öffnen kann, gilt als nicht abgegeben.

WAS INS REPO

Code, README (Setup + welches KI-Tool/Modell), saubere Commit-Historie — kein einzelner „Final“-Commit.

STAND ZUR FRIST

Es zählt der Stand zur Abgabefrist. Optional Commit-Hash/Tag. Keine Matrikelnummer ins Repo.

Voraussetzung: Einwilligung zur KI-Prüfung (vorige Folie). Ohne sie läuft die Abgabe händisch — ohne Nachteil.

Viel Erfolg. Bis nächste Woche.

SMART APPLICATIONS SOMMERSEMESTER 2026

DOZENT

Prof. Dr.-Ing. Jacek Zawisza

PROFESSUR

Betriebliche Informationssysteme & Digitalisierung · FB 3



HTW Corporate Design — Farbpalette.

Die verbindlichen CI-Farben mit Hex-Werten — für manuelle Anpassungen.

GRÜN-FAMILIE

	#4A7500 Dunkelgrün
	#76B900 HTW Grün
	#B5D96A Hellgrün
	#E8F5CC Hellgrün BG

ORANGE-FAMILIE

	#B23F00 Dunkelorange
	#FF5F00 Orange
	#FCB88A Hellorange
	#FFF3E8 Creme-Orange

NEUTRALE

	#1A1A1A Ink
	#333333 Grau
	#666666 Hellgrau
	#999999 Meta-Gray
	#D8D8D8 Rule
	#F5F5F5 Surface
	#FFFFFF Weiß
	#C7C9C3 Meta-Warm

DARK-MODE

	#0E0F0E Dark Deep
	#17181A Dark Base
	#19240F Dark Green Tint
	#1F2124 Dark Raised
	#2A2D31 Dark Elev
	#34373C Dark Rule
	#5A5C57 Dark Sub

Verbindlich: Primärfarbe HTW-Grün #76B900 · Akzente nur aus Grün- oder Orange-Familie · kein Blau.